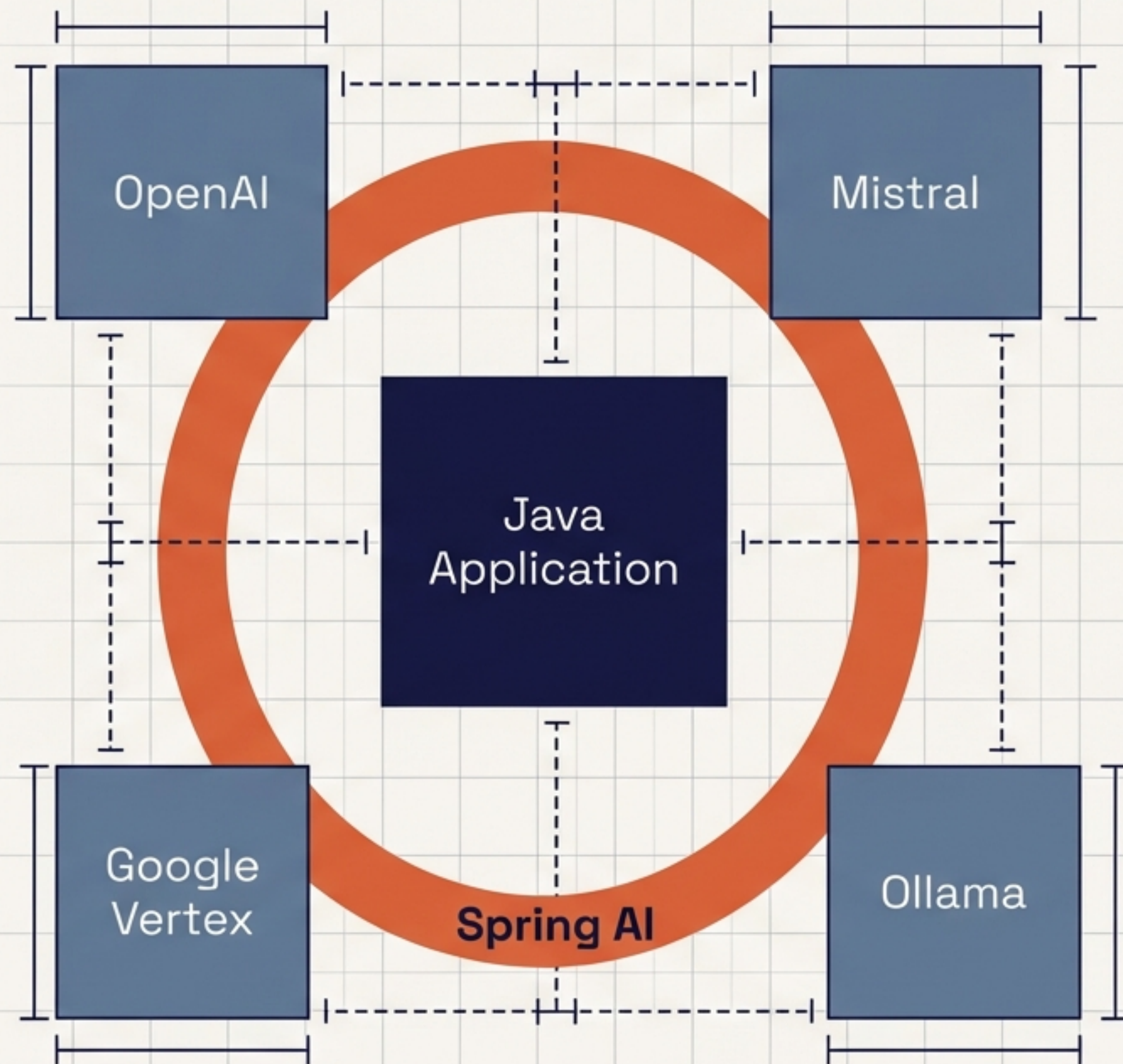


Construyendo componentes de Inteligencia Artificial nativos en Java

De conceptos fundacionales a orquestación de agentes complejos. Implementación práctica de RAG, Memoria y Tool Calling usando Spring AI.

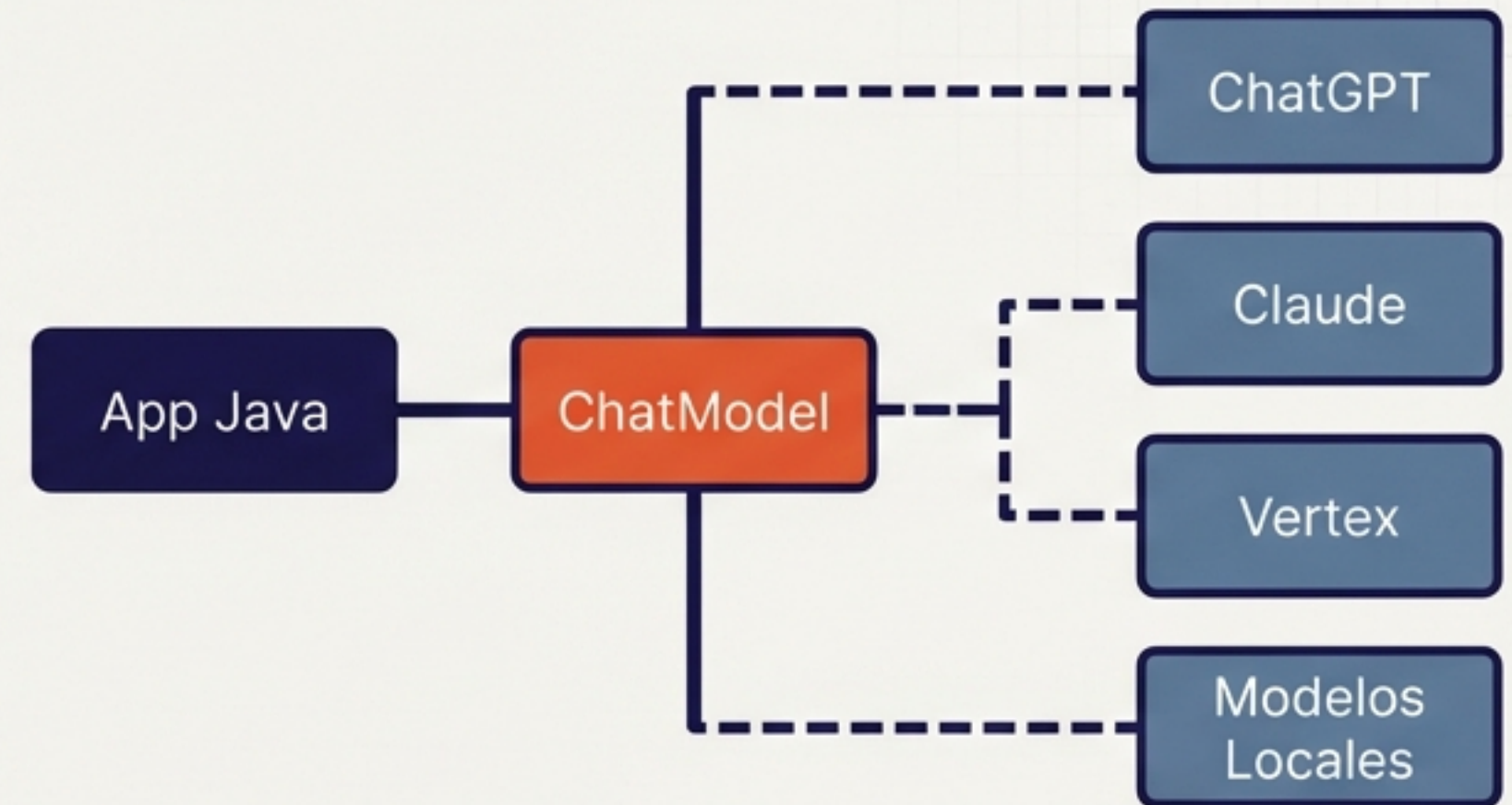


El adaptador universal para modelos fundacionales

Integración Tradicional de IA



El Enfoque Spring AI



Abstracción Portátil

Ejecuta el mismo código con ChatGPT, Claude o modelos locales en Ollama.

Filosofía Spring

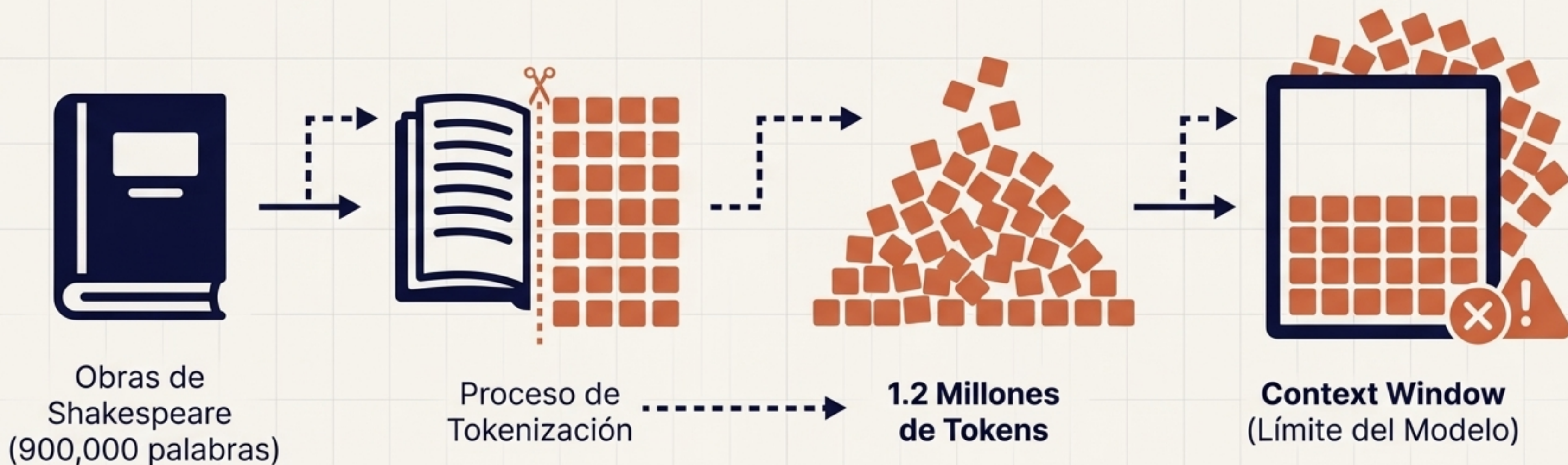
Principios de diseño modulares, inyección de dependencias y POJOs.

Integración de Datos

El puente definitivo entre los datos de tu empresa y los algoritmos de IA.

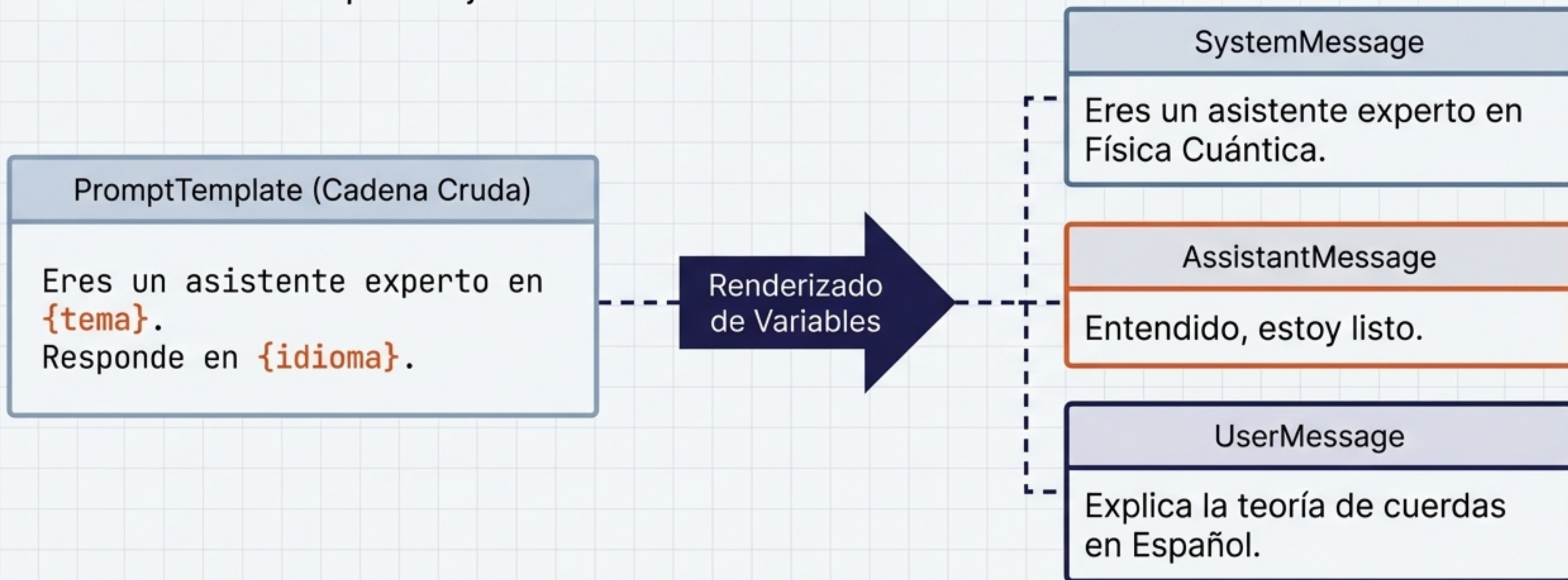
La materia prima cognitiva: Modelos y Tokens

- **Modelos:** Algoritmos pre-entrenados diseñados para predecir y generar información.
- **Tokens:** Los bloques de construcción básicos. 1 Token $\approx$ 0.75 palabras.
- **La Limitación Física:** La Ventana de Contexto restringe la cantidad de texto procesable por llamada y dicta el costo monetario.

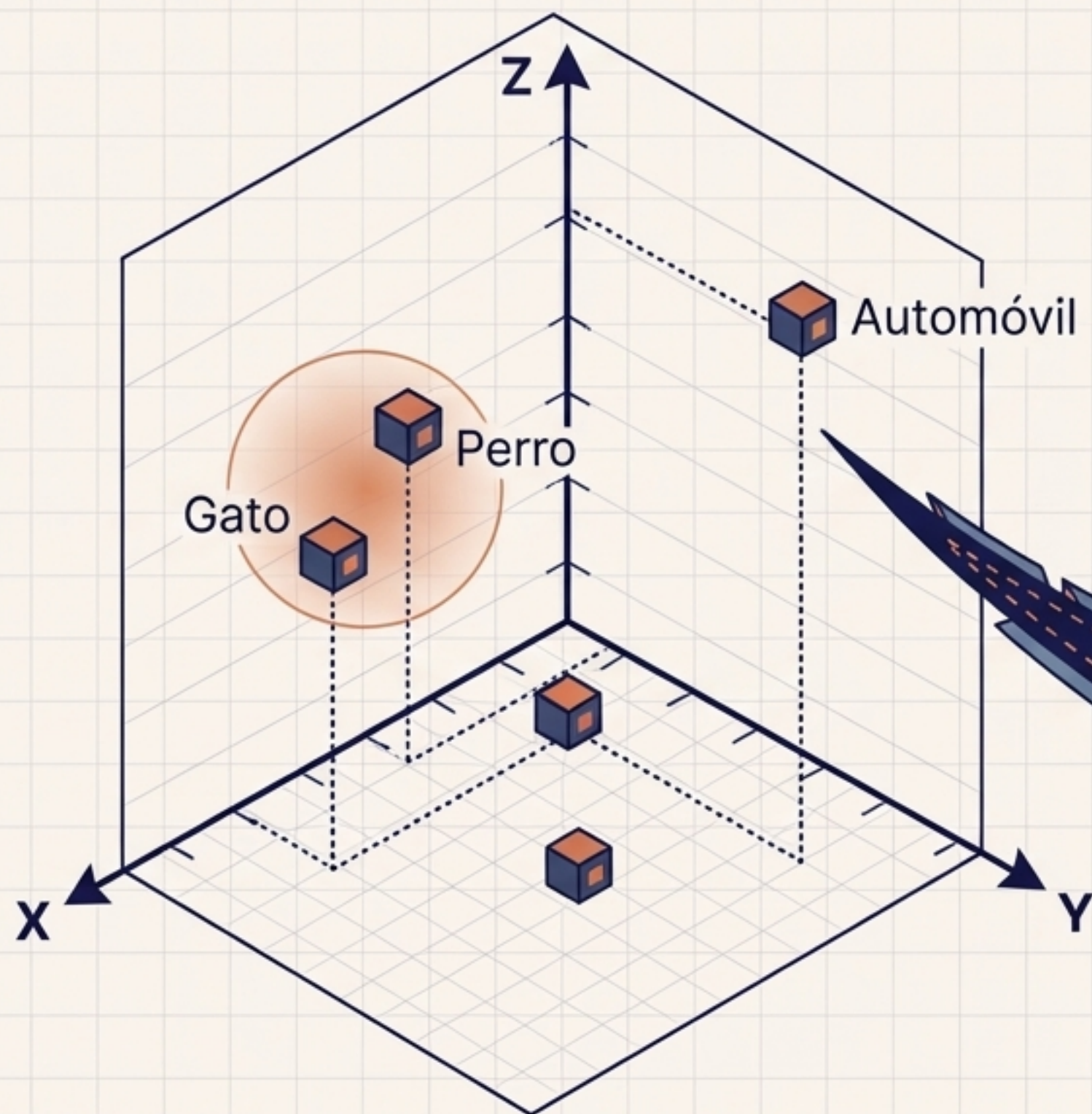


Orquestando instrucciones dinámicas con Prompts y Roles

- **Más que texto:** Un Prompt es una estructura de mensajes categorizados por roles.
- **El Motor de Plantillas:** Spring AI utiliza PromptTemplate para sustituir dinámicamente variables en tiempo de ejecución.



Embeddings: El mapa matemático de la comprensión semántica



- **Vectores:** Conversión de texto en matrices de números de punto flotante.
- **Proximidad = Similitud:** En este espacio de alta dimensión, la cercanía geométrica equivale a similitud de significado.
- **El Motor de Búsqueda de la IA:** Base fundamental para clasificación, búsqueda semántica y la Arquitectura RAG.



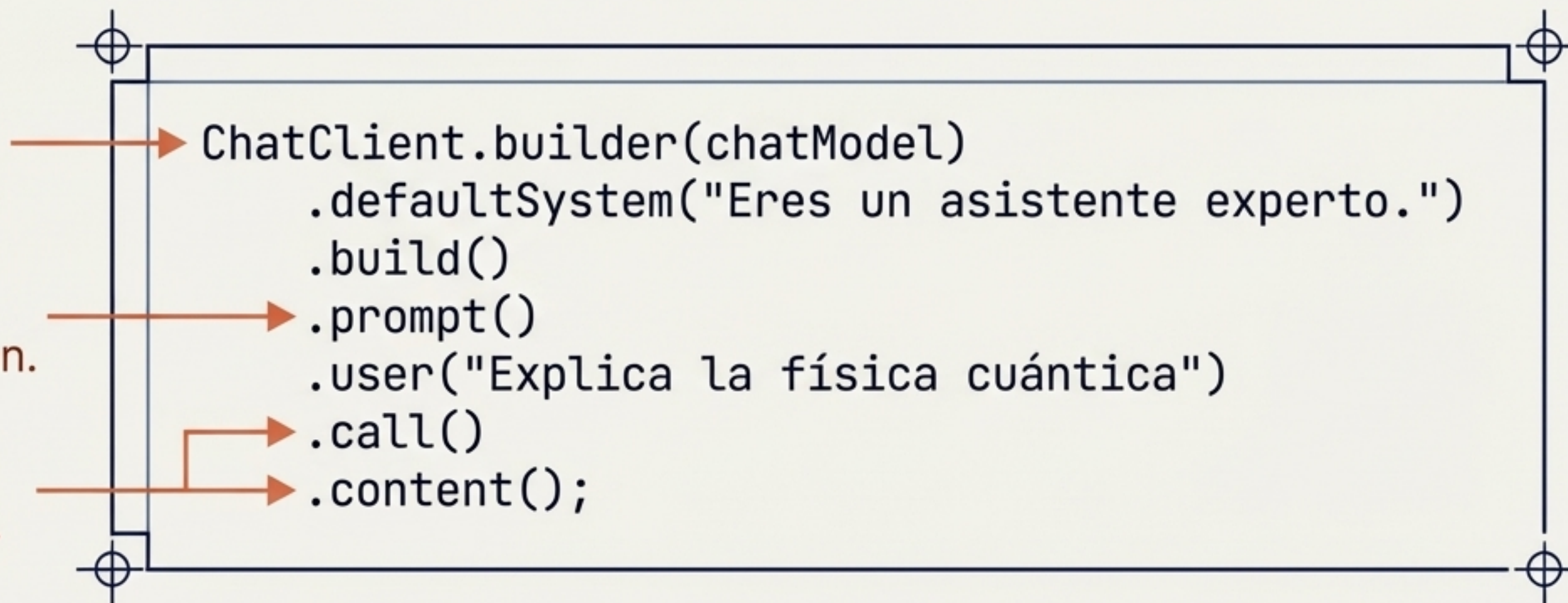
ChatClient: El comunicador fluido y universal

- **Diseño Fluido:** Una API idiomática similar a WebClient.
- **Modularidad:** Configura opciones por defecto en el arranque y sobrescríbelas en tiempo de ejecución.
- **Versatilidad:** Soporte para respuestas síncronas o reactivas devolviendo un Flux.

1. Inyección
y configuración base.

2. Construcción
dinámica de la petición.

3. Ejecución síncrona
y extracción de texto.



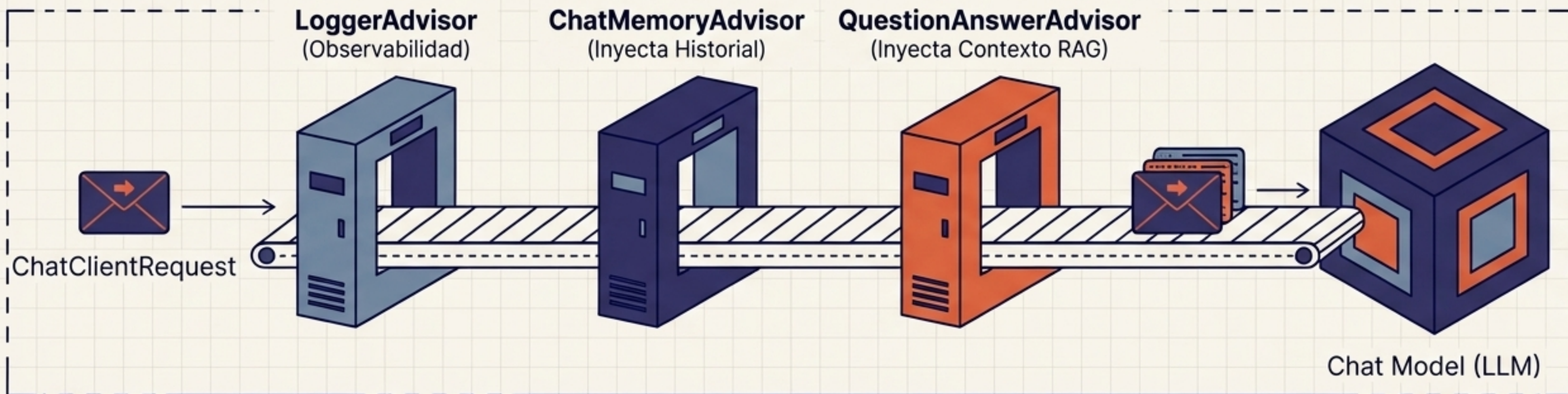
Structured Output: De texto probabilístico a Java Records

- ♦ **El Problema:** Extraer datos estructurados de salidas en lenguaje natural es frágil.
- ♦ **La Solución Spring AI:** Inyecta el esquema JSON al prompt y mapea automáticamente la respuesta a tu clase.
- ♦ **Tipado Fuerte:** Transforma la IA en un flujo de ingeniería de software predecible.



Advisors: Interceptores inteligentes en el ciclo de vida

- **Patrones Reutilizables:** Encapsulan lógicas recurrentes de IA generativa.
- **Ejecución en Cadena:** Modifican el Prompt antes de que llegue al modelo, y pueden interceptar la respuesta.
- **Portabilidad Máxima:** Los Advisors son agnósticos al modelo subyacente.



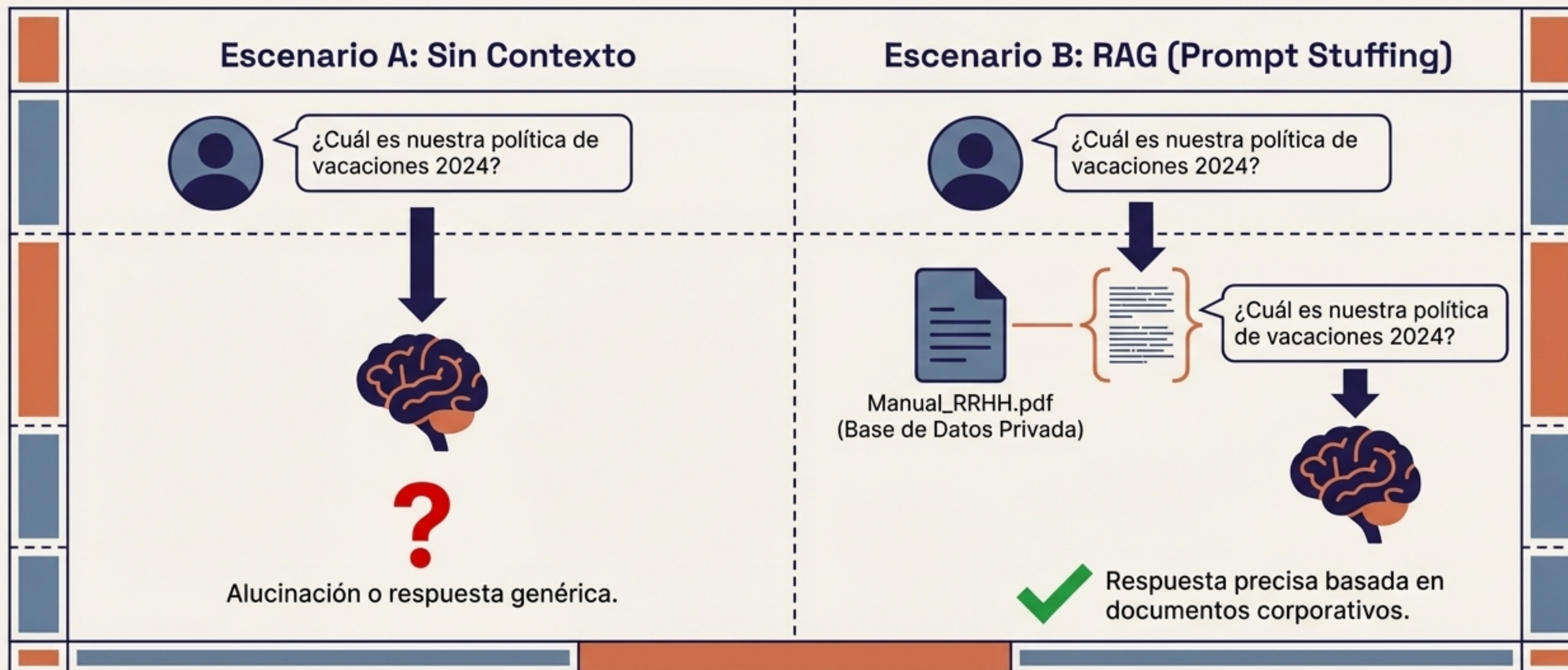
Gestión de Memoria: Curando la amnesia del modelo

- ♦ **La Naturaleza Sin Estado:** Los LLMs no recuerdan interacciones pasadas; el contexto debe ser reenviado.
- ♦ **MessageWindowChatMemory:** Mantiene solo los N mensajes más recientes, descartando los más antiguos.
- ♦ **Optimización de Tokens:** Preserva las instrucciones del sistema mientras evita desbordar la ventana de contexto y controla costos.



El desafío de la privacidad y las alucinaciones

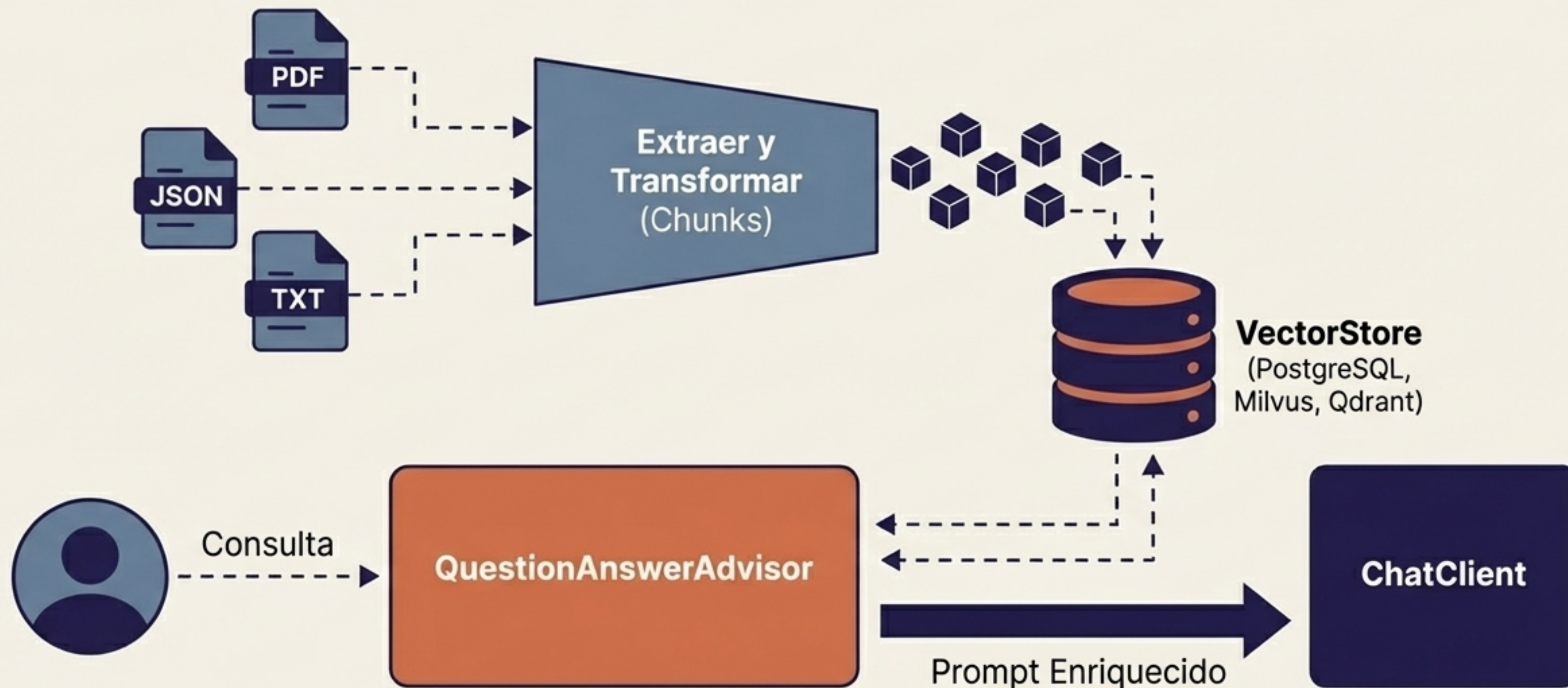
- **Las Limitaciones de los Modelos:** Tienen conocimiento obsoleto, sufren de alucinaciones al adivinar, y no tienen acceso a tus bases de datos.
- **Ajuste Fino vs. RAG:** Entrenar modelos es caro. La solución es Rellenar el Prompt dinámicamente con información relevante en tiempo de ejecución.



Arquitectura RAG: Tuberías ETL y Vector Databases

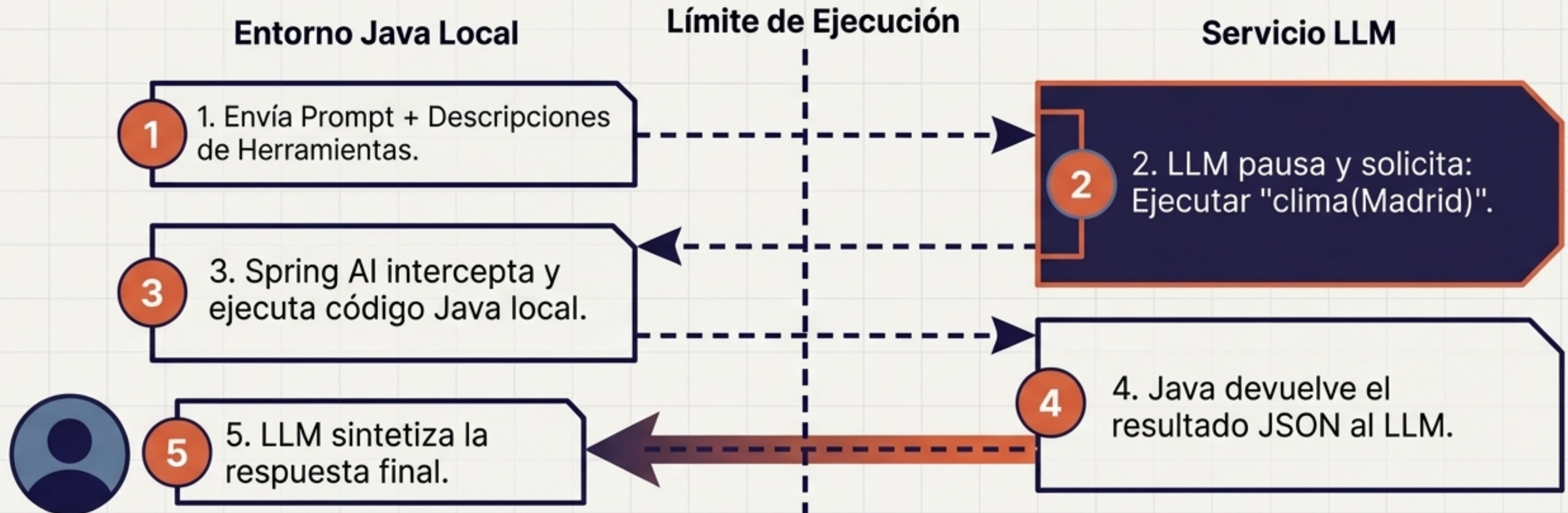
Fase 1 (Ingesta - ETL): Lectura de documentos no estructurados, transformación en embeddings y carga en almacenes vectoriales.

Fase 2 (Recuperación - Advisor): Búsqueda de similitud semántica y enriquecimiento del Prompt con cero código repetitivo.



Tool Calling: Otorgando capacidad de acción al Agente

- **Rompiendo el aislamiento:** Los LLMs no pueden ejecutar código ni hacer llamadas HTTP por sí mismos.
- **Inversión de Control:** El modelo determina cuándo y con qué parámetros ejecutar una función, pero la ejecución real ocurre de manera segura en Java.



Transformando código en capacidades con @Tool

- **Anotaciones Nativas:** Usa @Tool en métodos estándar.
- **La Importancia de las Descripciones:** La instrucción primaria que el modelo usa para decidir si debe usar la herramienta.
- **Inferencia Dinámica:** Generación automática de esquemas JSON basados en los tipos de datos.

Código Java

```
@Tool(name="getWeather", -----  
description="Obtiene el clima actual para  
una ciudad")  
public WeatherResponse getCityWeather(  
    @ToolParam(description="Nombre de la  
ciudad") String city) {  
    // Lógica local...  
}
```

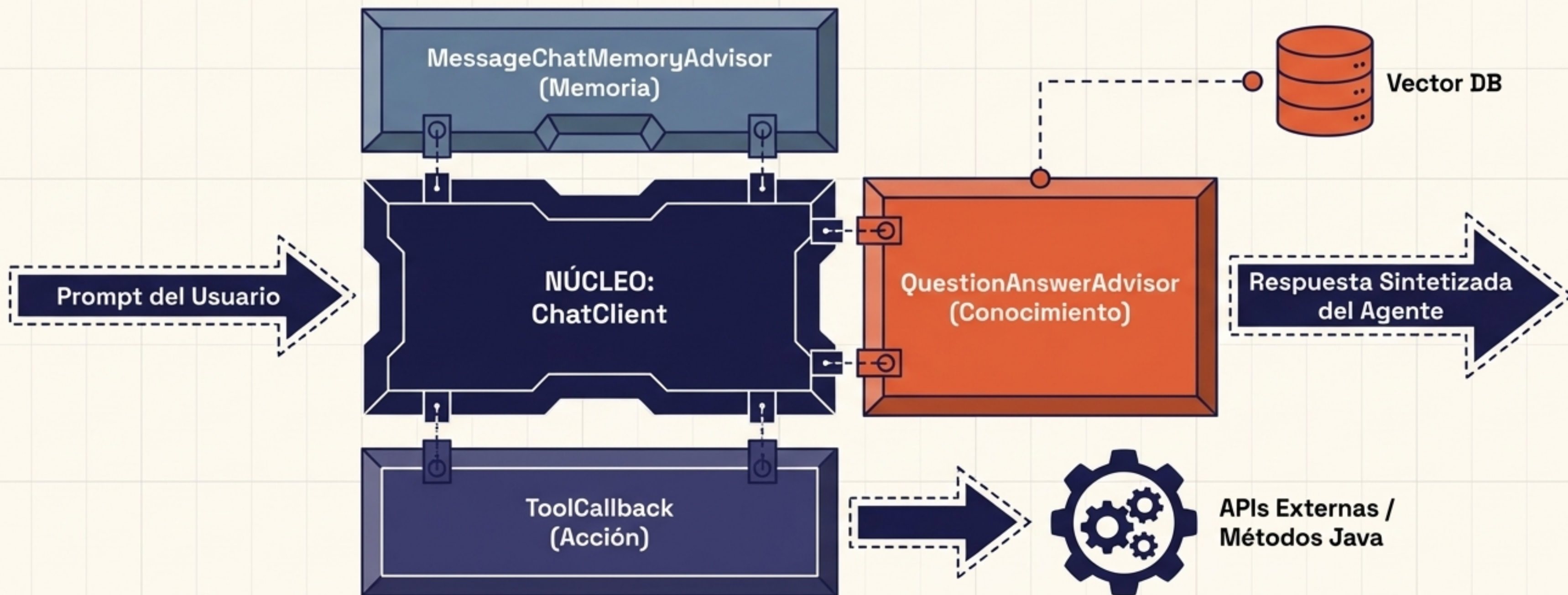
Introspección
de Spring

Esquema Generado

```
{  
  "type": "function",  
  "function": {  
    "name": "getWeather",  
    "description": "Obtiene el clima...",  
    "parameters": { "city": "string" }  
  }  
}
```


Síntesis Estructural: Anatomía de un Agente Inteligente

- **Orquestación Unificada:** Un único ciclo de ChatClient maneja contexto histórico, recuperación dinámica y ejecución de funciones externas.
- **Ingeniería Predictiva:** Convertimos la aleatoriedad de los LLMs en conductos de software estructurados y fiables.



Ya conoces Java. Ahora conoces la IA.

- **Spring AI** elimina el código repetitivo de integración de LLMs, bases de datos vectoriales y orquestación de prompts.
- **Desarrolla** aplicaciones de Inteligencia Artificial utilizando las mismas convenciones y abstracciones que usas todos los días.
- **Transforma** tu arquitectura tradicional en un ecosistema inteligente hoy mismo.

Inicializa tu proyecto

start.spring.io



Introspección
de Spring

Explora la documentación

Guía de Referencia Spring AI

